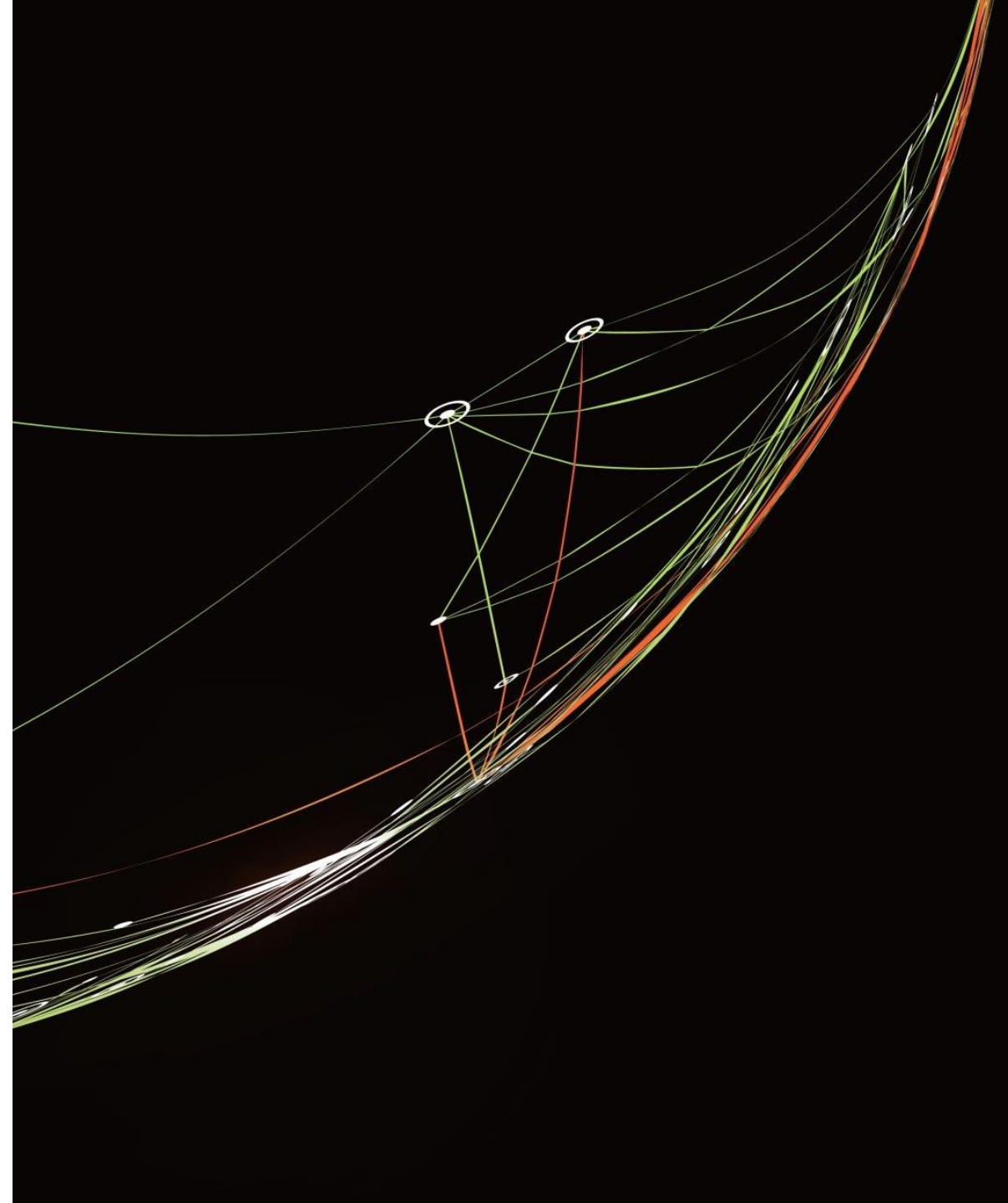

Welcome to PrivML-FHE

IMPLEMENTATION OF A PRIVACY-
PRESERVING MACHINE LEARNING MODEL
USING HOMOMORPHIC ENCRYPTION

By Thomas Green
Supervisor: Arman Khouzani



Motivation

COLLECT DATA -> TRAIN MODELS -> DEPLOY -> BUSINESS VALUE

Machine learning has brought significant advancements through data-driven innovation, relying on large volumes of data for effective training and using user data for inference.

Users and organisations increasingly depend on ML to extract insights and decision-making.

The Current Infrastructure

- Models trained on billions of data points.
- Personal health records, financial data, etc.
- Third-party cloud providers, international jurisdictions.

Growing Concerns

- Regulatory Landscape
- Public Awareness
- Market Pressure



The Problem

DECRYPT -> PROCESS -> INFERENCE -> ENCRYPT

Why This Matters?

- Exposure to breaches
- Regulatory and Legal Risks
- Architectural Limitations

The question is not whether ML systems are valuable

The question is can we redesign ML architectures to eliminate the decryption requirement?



The Solution

~~DECRYPT~~ → PROCESS → INFERENCE → ~~ENCRYPT~~
ENCDATA → PROCESS → INFERENCE → ENCRESULTS

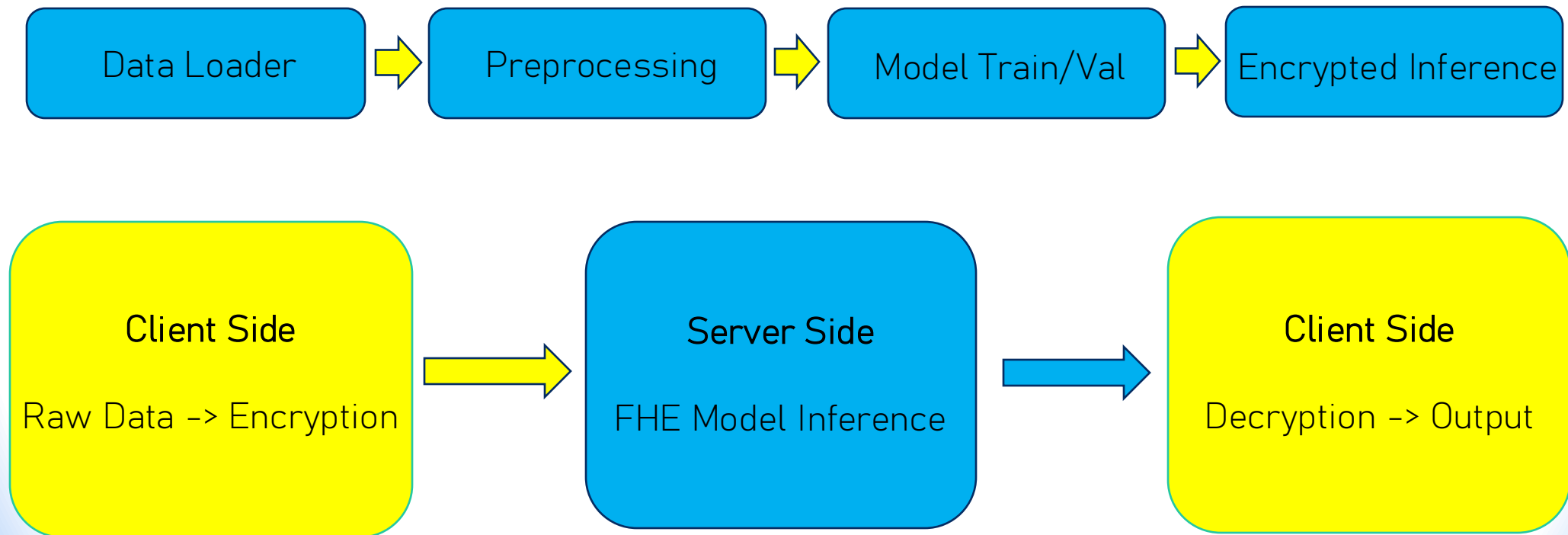
Fully Homomorphic Encryption (FHE)

Enables computation directly on encrypted data
No need to decrypt at any point!

- Mathematical security guarantee
- End-to-end confidentiality
- Zero-trust compatibility
- Secure inference
- Reduced legal and ethical risks



Design



Implementation

FHE Library Research

- Concrete-ML
- TenSEAL
- PySEAL

Evaluation Script

- Classification, Regression Metrics, Time, Resource Usage
- Logs data from training, validation, testing and FHE inference.
- Continuously appends to .csv and generates .pdf with model outputs + plots

Hyperparameter Tuning

- Configuration file set for each ML+FHE pipeline for easy model tuning.
- Explored different ML parameters; activation functions, loss functions, learning rates, hidden dimension size, number of epochs.
- Explored different FHE parameters; polynomial modulus degree, coefficient modulus bits.



Implementation

Number Detector

Classification

Input: 28x28 (Greyscale)

Output: 10 Classes (0-9)

Architecture: CNN



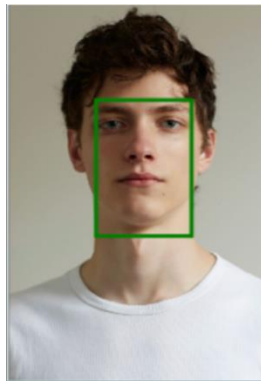
Face Detection

Regression

Input: 128x128 (RGB)

Output: Bounding Box

Architecture: CNN



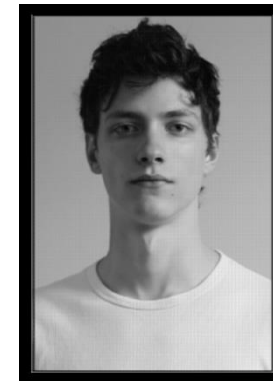
Border Generation

Regression

Input: 32x32 (Grayscale)

Output: Augmented Input Image

Architecture: CNN



FHE Scheme: CKKS (supports approximate arithmetic on real numbers)

Security Level: 128-256 bit

Framework: PyTorch -> TenSEAL FHE Wrapper

Implementation

- Built on HTML, CSS, JavaScript. Utilizes Node.js
- Python backend for data encryption and model loading
- Two servers acting as trusted and untrusted domains to simulate real-world deployment
- Three pages for each ML model, with both developed plaintext and FHE model
- Serves as an evaluation tool between non-encrypted and encrypted models



Implementation

- Built on HTML, CSS, JavaScript. Utilizes Node.js
- Python backend for data encryption and model loading
- Two servers acting as trusted and untrusted domains to simulate real-world deployment
- Three pages for each ML model, with both developed plaintext and FHE model
- Serves as an evaluation tool between non-encrypted and encrypted models

Encrypted – FHE

Step One

Upload Face Image

Insert Image

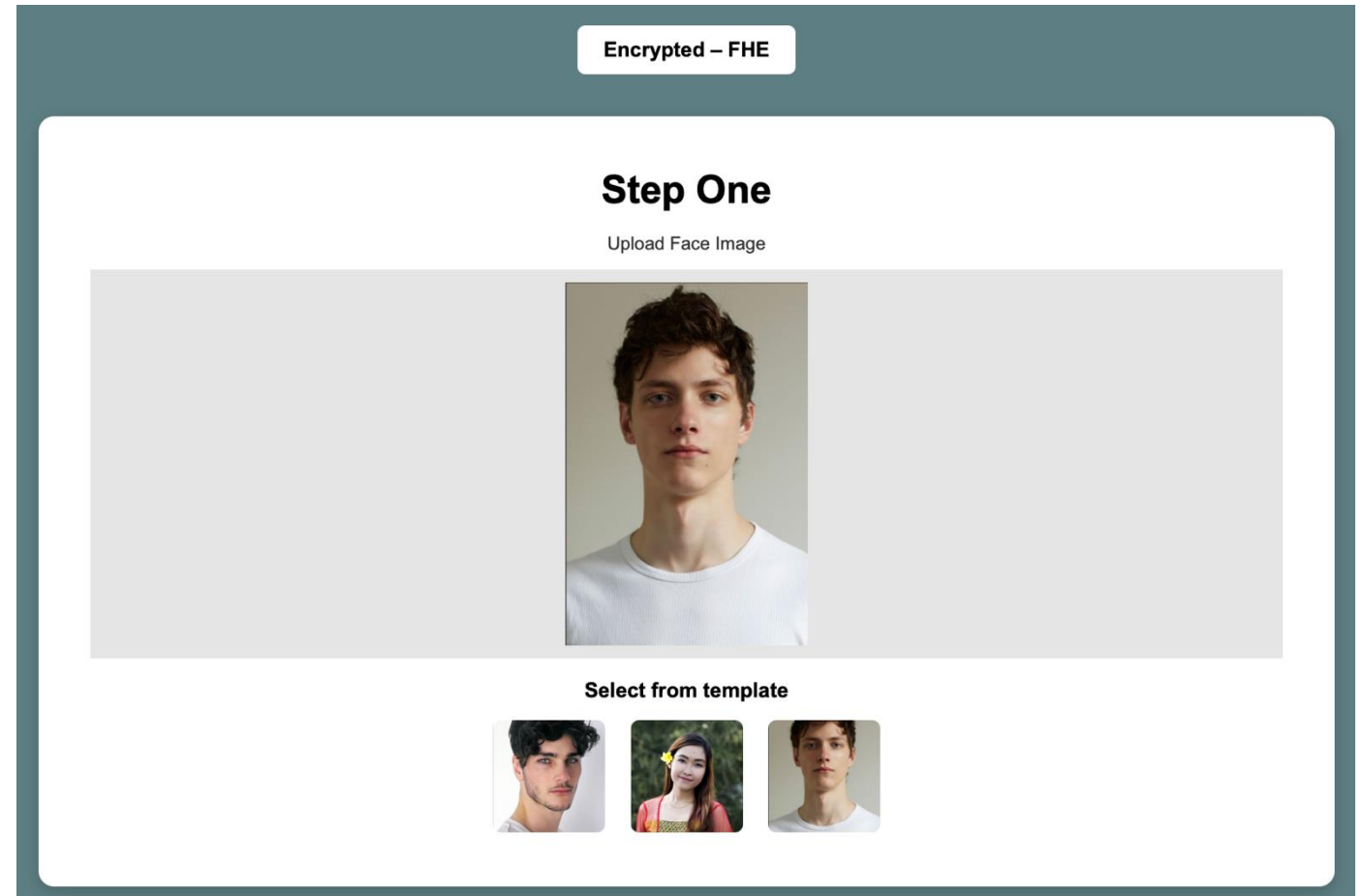
Drag & drop image here

Select from template



Implementation

- Built on HTML, CSS, JavaScript. Utilizes Node.js
- Python backend for data encryption and model loading
- Two servers acting as trusted and untrusted domains to simulate real-world deployment
- Three pages for each ML model, with both developed plaintext and FHE model
- Serves as an evaluation tool between non-encrypted and encrypted models



Implementation

- Built on HTML, CSS, JavaScript. Utilizes Node.js
- Python backend for data encryption and model loading
- Two servers acting as trusted and untrusted domains to simulate real-world deployment
- Three pages for each ML model, with both developed plaintext and FHE model
- Serves as an evaluation tool between non-encrypted and encrypted models

The image shows a UI mockup with two distinct sections, each enclosed in a light gray rounded rectangle with a dark gray border. The top section is titled 'Step Two' in bold black text, with the subtitle 'Enter Password To Encrypt Image' below it. It contains a text input field with the placeholder 'Enter password...' and a dark gray button labeled 'Encrypt'. The bottom section is titled 'Step Three' in bold black text, with the subtitle 'Encrypted Image Preview' below it. It features a large, empty light gray rectangular area for image preview and a dark gray button at the bottom right labeled 'Send Image to AI'.

Step Two

Enter Password To Encrypt Image

Encrypt

Step Three

Encrypted Image Preview

Send Image to AI

Implementation

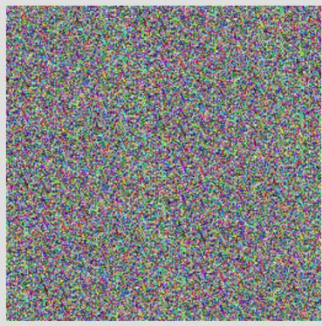
- Built on HTML, CSS, JavaScript. Utilizes Node.js
- Python backend for data encryption and model loading
- Two servers acting as trusted and untrusted domains to simulate real-world deployment
- Three pages for each ML model, with both developed plaintext and FHE model
- Serves as an evaluation tool between non-encrypted and encrypted models

Step Two

Enter Password To Encrypt Image

Step Three

Encrypted Image Preview



Implementation

- Built on HTML, CSS, JavaScript. Utilizes Node.js
- Python backend for data encryption and model loading
- Two servers acting as trusted and untrusted domains to simulate real-world deployment
- Three pages for each ML model, with both developed plaintext and FHE model
- Serves as an evaluation tool between non-encrypted and encrypted models

Log Outputs

```
[INFO] Waiting for image...
```

Step Four

Enter Password To Decrypt Output

Decrypt

Implementation

- Built on HTML, CSS, JavaScript. Utilizes Node.js
- Python backend for data encryption and model loading
- Two servers acting as trusted and untrusted domains to simulate real-world deployment
- Three pages for each ML model, with both developed plaintext and FHE model
- Serves as an evaluation tool between non-encrypted and encrypted models

Log Outputs

```
[INFO] Waiting for image...
[INFO] Reconstructing TenSEAL Data...
[INFO] Loading face detection model...
[INFO] Encrypted model instance created.
[INFO] Running encrypted inference...
[INFO] Inference complete in 9.329s
[INFO] Writing encrypted output.
[INFO] Received encrypted face result. Ready to decrypt.
```

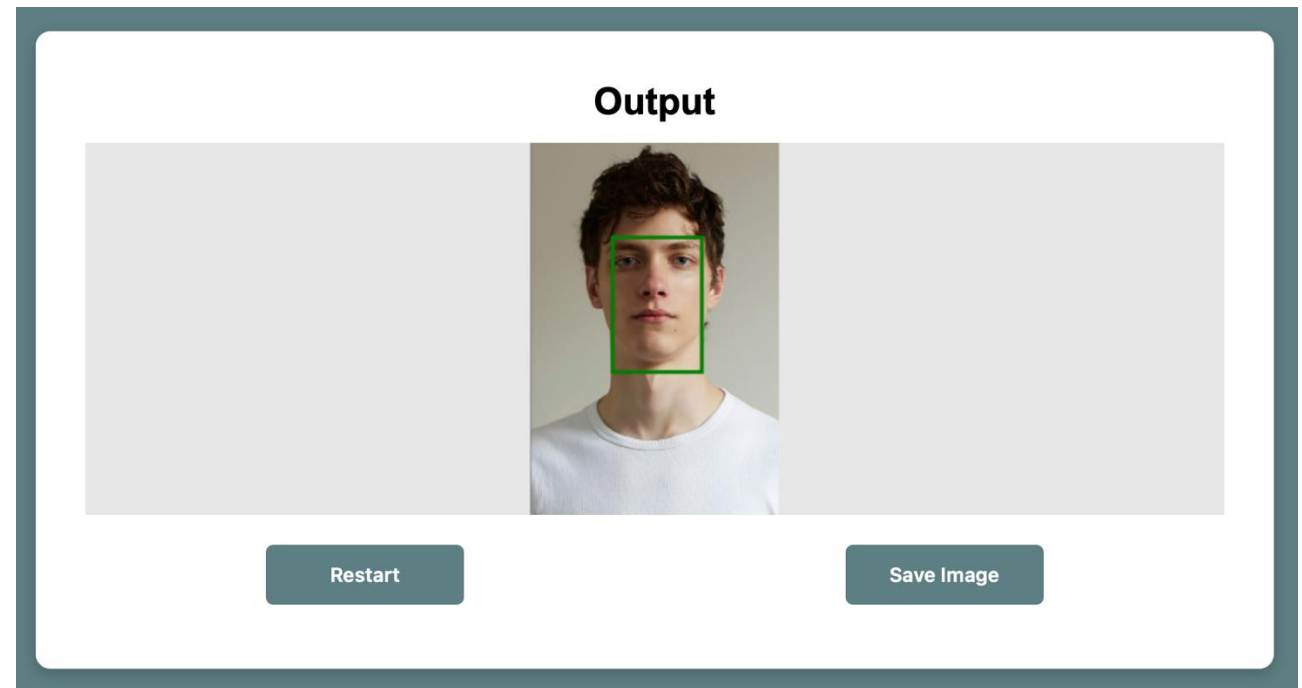
Step Four

Enter Password To Decrypt Output

Decrypt

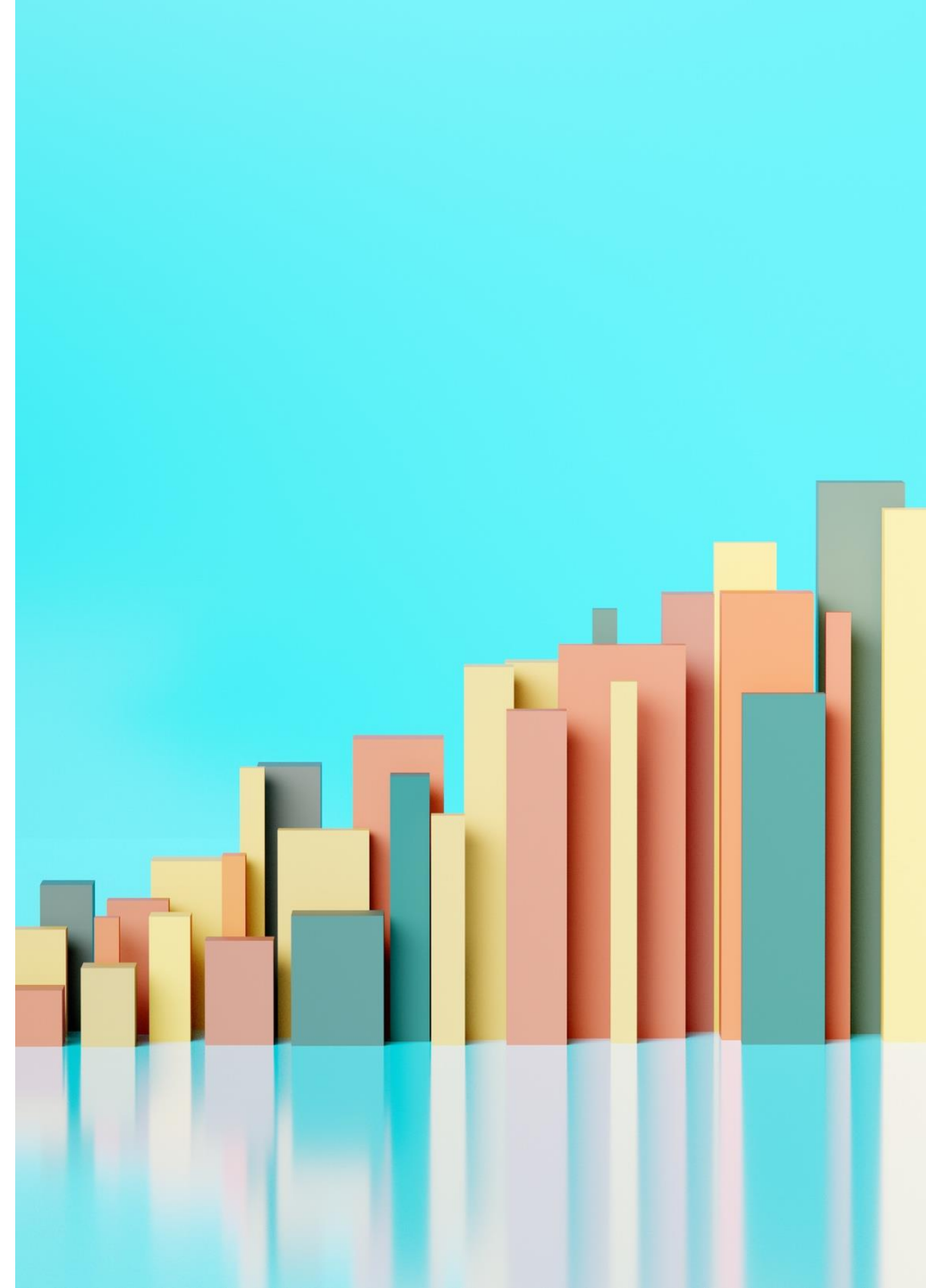
Implementation

- Built on HTML, CSS, JavaScript. Utilizes Node.js
- Python backend for data encryption and model loading
- Two servers acting as trusted and untrusted domains to simulate real-world deployment
- Three pages for each ML model, with both developed plaintext and FHE model
- Serves as an evaluation tool between non-encrypted and encrypted models



Evaluation

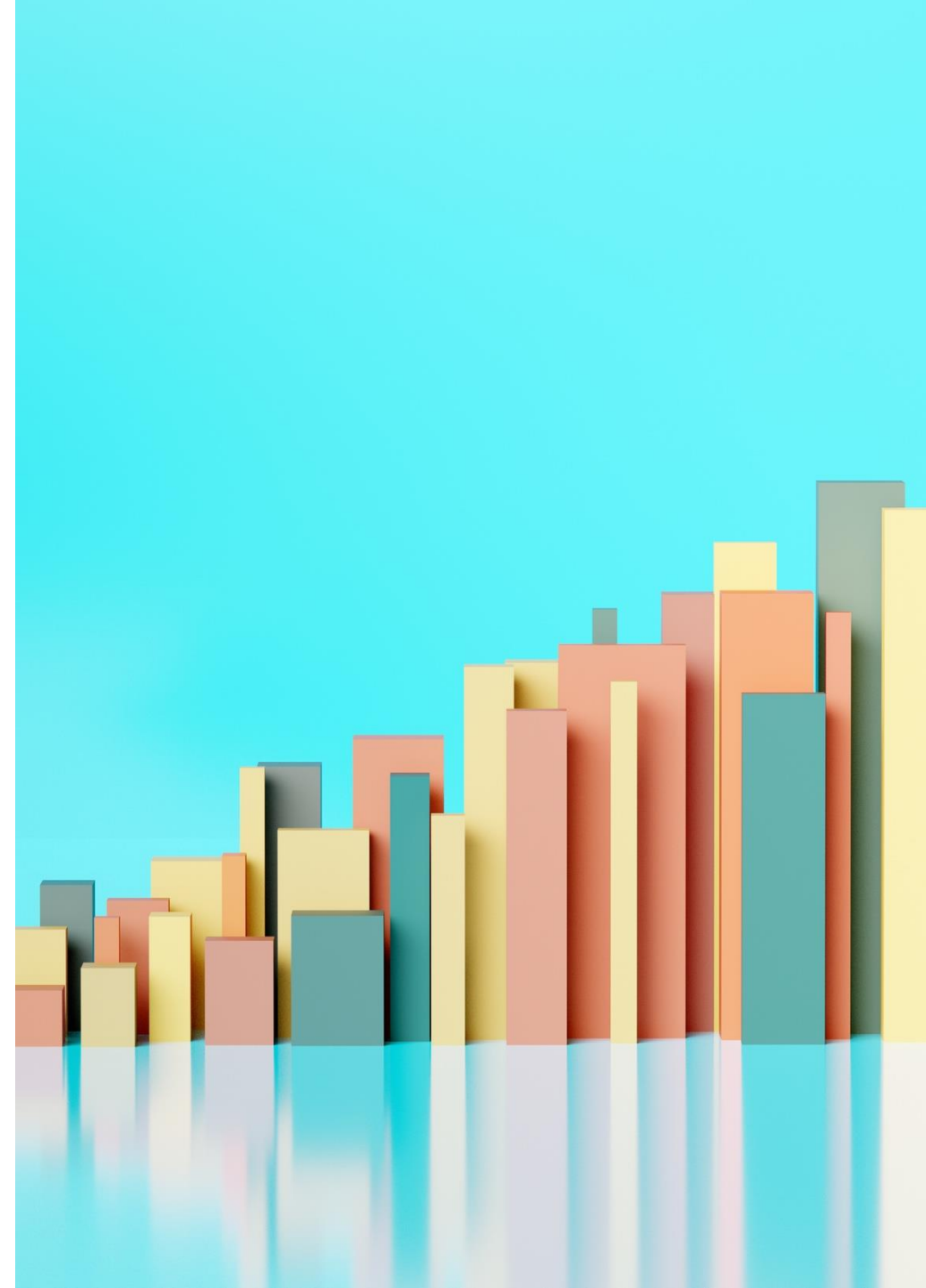
- User Testing -> Conducted system usability scale (SUS) testing.
- Model Evaluation
 - Accuracy comparison: Model performance on encrypted vs. plaintext data
 - Processing time: Training/Inference time with FHE
 - Memory usage: RAM and CPU consumption and ciphertext expansion factors



Results

Pipeline	Plaintext Accuracy/MSE	Encrypted Accuracy/MSE	Inference Time
MNIST Classification	97.0% accuracy	97.0% accuracy	1.6s
Face Detection	0.0073 MSE	0.0036 MSE*	9.3s
Border Generation	0.0541 MSE	270.21 MSE†	0.26s

- SUS Score: 82.97
- Model Evaluation
 - Encrypted performance better due to CKKS scheme approximation and smoothing
 - Computation overhead identified 5.95–8.5GB for encrypted inference.
 - Architecture constraints: Limited to 2 conv layers due to multiplicative depth.





Findings and Future Work

- **Feasibility Proven:** FHE-ML achievable with current technology but requires careful architectural adaptation.
- **Classification vs Regression:** Classification tasks maintain accuracy under encryption; regression suffers from noise accumulation.
- **Performance Bottlenecks:** Input resolution and model depth drive computational overhead and noise growth.
- **TenSEAL Limitations:** No bootstrapping support limits multiplicative depth; restricted layer compatibility requires custom implementation

Critical Trade-offs

- Model complexity vs. FHE feasibility
- Privacy guarantees vs computational cost
- Accuracy vs Architectural simplifications



Findings and Future Work

Future Work

- **Bootstrapping** support to enable deeper networks without depth constraints.
- **FHE wrapper toolkit** for automated PyTorch-to-FHE model conversion
- **Federated Learning + FHE** for collaborative privacy-preserving training
- **Hardware acceleration** to address computational overhead

Sustainability Considerations

- **Environmental:** Energy costs require hardware optimization
 - **Social:** Enables ethical and privacy-preserving AI in business sectors
 - **Economic:** New market opportunities
- 